

Hot 100 总结

1. 两数之和

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那 **两个** 整数，并返回它们的数组下标。

输入: `nums = [2,7,11,15]`, `target = 9`
输出: `[0,1]`
解释: 因为 `nums[0] + nums[1] == 9`，返回 `[0, 1]`。

判断字典中是不是有这个key 用 `a in dict` 即可

```
def twoSum(self, nums: List[int], target: int) -> List[int]:
    dic = {}
    n = len(nums)
    dic[nums[0]] = 0
    for i in range(1, n):
        if dic.get(target - nums[i], -1) != -1:
            return [i, dic[target - nums[i]]]
        else:
            dic[nums[i]] = i
```

2. 字母异位分组

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

输入: `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`
输出: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

```
def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
    dic = {}
    for i in strs:
        i_sort = "".join(sorted(i))
        if i_sort in dic:
            dic[i_sort].append(i)
        else:
            dic[i_sort] = [i]
    return list(dic.values())
```

3. 最长连续序列

给定一个未排序的整数数组 `nums`，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

输入: `nums = [100,4,200,1,3,2]`
输出: 4
解释: 最长数字连续序列是 `[1, 2, 3, 4]`。它的长度为 4。

使用set 一遍list 时间复杂度为n set底层是hash表实现 dict也是

```
def longestConsecutive(self, nums: List[int]) -> int:
    newnums = set(nums)
    ans = 0
    for i in newnums:
        # 为了保证时间复杂度 只有在重新算时才会进入内层循环 因为判断i+1的下次还会碰到
        if i-1 not in newnums:
            current_num = i
            current = 1
            while current_num + 1 in newnums:
                current+=1
                current_num +=1
            ans = max(ans, current)
    return ans
```

4. 移动零

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

输入: `nums = [0,1,0,3,12]`

输出: `[1,3,12,0,0]`

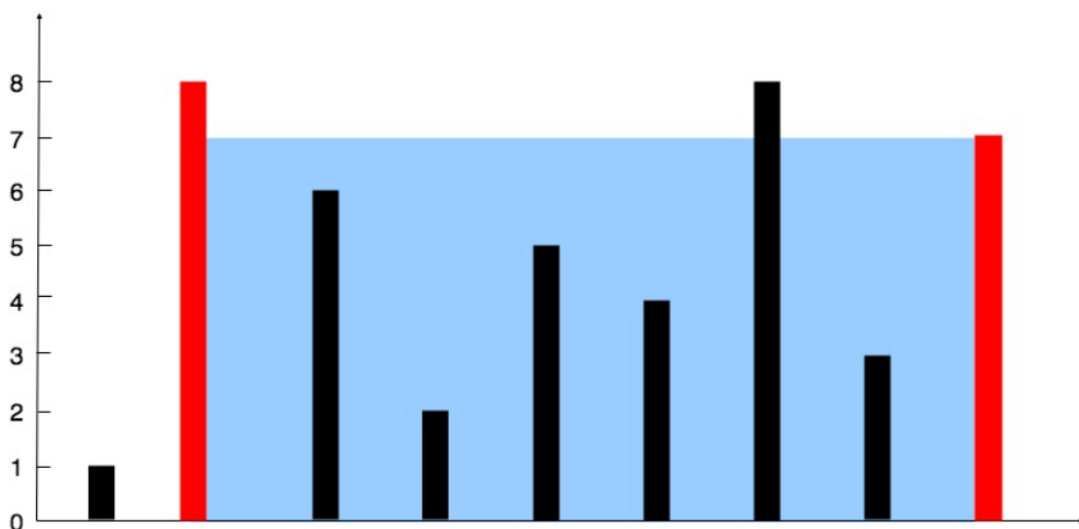
```
def moveZeroes(self, nums: List[int]) -> None:
    l1 = l2 = 0
    n = len(nums)
    # l1 指向0 除了开始时可能不是 l2代表即将要处理的位置
    # l1是比l2慢的 也就是说l1如果没和l2一起 那就是指向0的
    while l2 < n:
        if nums[l2] != 0:
            nums[l2], nums[l1] = nums[l1], nums[l2]
            l1+=1
        l2+=1
```

5. 盛最多水的容器

给定一个长度为 `n` 的整数数组 `height`。有 `n` 条垂线，第 `i` 条线的两个端点是 `(i, 0)` 和 `(i, height[i])`。

找出其中的两条线，使得它们与 `x` 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。



输入: [1,8,6,2,5,4,8,3,7]

输出: 49

解释: 图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下, 容器能够容纳水(表示为蓝色部分)的最大值为 49。

正常双指针

```
def maxArea(self, height: List[int]) -> int:
    left = 0
    right = len(height)-1
    ans = 0
    while left<right:
        ans = max(ans, min(height[left],height[right])*(right-left))
        if height[left] < height[right]:
            left+=1
        elif height[left] > height[right]:
            right-=1
        else:
            left+=1
            right-=1
    return ans
```

6. 三数之和

给你一个整数数组 `nums`, 判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k`, 同时还满足 `nums[i] + nums[j] + nums[k] == 0`。请你返回所有和为 0 且不重复的三元组。

输入: `nums = [-1,0,1,2,-1,-4]`

输出: `[[-1,-1,2],[-1,0,1]]`

解释:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`。

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`。

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`。

不同的三元组是 `[-1,0,1]` 和 `[-1,-1,2]`。

注意, 输出的顺序和三元组的顺序并不重要。

先排序

关键之处在于去重

```
def threeSum(self, nums: List[int]) -> List[List[int]]:
    nums.sort()
    n = len(nums)
    ans = []
    # 去重: 除了入循环第一次不计入考虑
    for i in range(n-2):
        if i > 0 and nums[i-1]==nums[i]:
            continue
        target = -nums[i]
        t = n-1
        for j in range(i+1,n-1):
            if j>i+1 and nums[j-1]==nums[j]:
                continue
            while j<t and nums[j]+nums[t]>target:
```

```

        t-=1
    if j==t:
        break
    if nums[i]+nums[j]+nums[t]==0:
        ans.append([nums[i],nums[j],nums[t]])
return ans

```

7. 接雨水

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。



输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

单调栈 类似题目 栈模块的 每日温度等

```

def trap(self, height: List[int]) -> int:
    sstack = []
    n = len(height)
    ans = 0
    for i in range(n):
        while len(sstack)!=0 and height[i]>height[sstack[-1]]:
            mid = sstack.pop()
            if len(sstack)!=0:
                left = sstack[-1]
                ans = ans+(i-left-1)*(min(height[i],height[left])-height[mid])
            sstack.append(i)
    return ans

```

8. 无重复字符的最长子串

给定一个字符串 s ，请你找出其中不含有重复字符的 **最长子串** 的长度。

输入: s = "abcabcbb"

输出: 3

解释: 因为无重复字符的最长子串是 "abc"，所以其长度为 3。注意 "bca" 和 "cab" 也是正确答案。

滑动窗口，左指针指向最左面 右指针循环往后

集合操作 加入是add 删除是remove

确定左边界时要注意 while循环 因为重复的不一定是在最左面

```
def lengthOfLongestSubstring(self, s: str) -> int:
    sset = set()
    ans = 1 = 0
    for r in range(len(s)):
        while s[r] in sset:
            sset.remove(s[l])
            l+=1
        sset.add(s[r])
        ans = max(ans, r-l+1)
    return ans
```

9. 找到字符串中所有字母异位词

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

输入：s = "cbaebabacd", p = "abc"

输出：[0,6]

解释：

起始索引等于 0 的子串是 "cba"，它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac"，它是 "abc" 的异位词。

一个数组[0]*26 不好写 可以搞两个 分别统计s和p 比较的时候可以 直接==

注意 python3中比较两个列表是比较逐个元素

一个数组的就得初始化时加一 之后新加入减一了剔除的加一 判断是否全为0

```
def findAnagrams(self, s: str, p: str) -> List[int]:
    dic = [0]*26
    ans = []
    for i in p:
        dic[ord(i)-97]+=1
    left = 0
    right = len(p)-1
    t = True
    if right<len(s):
        for i in range(left, right+1):
            dic[ord(s[i])-97]-=1
        for key in dic:
            if key!=0:
                t = False
                break
        if t:
            ans.append(0)
        left+=1
        right+=1
    while right<len(s):
        t = True
        dic[ord(s[left-1])-97]+=1
        dic[ord(s[right])-97]-=1
        for key in dic:
            if key!=0:
```

```

        t = False
        break
    if t:
        ans.append(left)
        left+=1
        right+=1
    return ans

```

判断子串是否包含了目标串 有一个题与此有异曲同工之处 --- 最小覆盖子串

10. 和为K的子数组###

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。

子数组是数组中元素的连续非空序列。

输入: `nums = [1,1,1]`, `k = 2`
 输出: 2

考察点 前缀和

为什么`dic[chazhi] = 多少`就可以直接加多少

看一个例子:

1, 2, -1, -2, 1, 2, 1 和为4

1, 3, 2, 0, 1, 3, 4 到最后时有两个地方和为3 所以最终和为4的有两种组合 相当于前面先为3的是包含了后为3的子串

```

def subarraySum(self, nums: List[int], k: int) -> int:
    s = 0
    dic = collections.defaultdict(int)
    dic[0]=1 # 防止刚开始第一个数就等于k
    ans = 0
    for i in nums:
        s+=i
        chazhi = s-k
        if dic[chazhi] !=0:# dic有个几个 就代表前面前缀和为chazhi的有几个
            ans += dic[chazhi] #和为chazhi的位置到和为s的位置就是和为k的子串
        dic[s]+=1
    return ans

```

11. 滑动窗口最大值 重点

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 滑动窗口中的最大值。

输入: nums = [1,3,-1,-3,5,3,6,7], k = 3

输出: [3,3,5,5,6,7]

解释:

滑动窗口的位置	最大值
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

问题的关键就是要确定移动之后原先的最大值还在不在 该不该在

方法一: 单调双端队列

队头始终是最大值

每次滑动后 队尾比不过新来的通通干掉 (被干掉的 现在不是最大值 以后更不可能) 队头超出滑动窗口干掉

```
def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
    deque = collections.deque()
    ans = []
    n = len(nums)
    for i in range(k):
        while deque and nums[i] >= nums[deque[-1]]:
            deque.pop()
        deque.append(i)
    ans.append(nums[deque[0]])
    for j in range(k, n):
        while deque and nums[j] >= nums[deque[-1]]:
            deque.pop()
        deque.append(j)
        if deque[0] <= j - k:
            deque.popleft()
        ans.append(nums[deque[0]])
    return ans
```

方法二 堆

建堆(默认小根堆) heapify(list) heappush加入 heappop()删除一个

```
ans = []
q = [(-nums[i], i) for i in range(k)]
heapify(q)
ans.append(-q[0][0])
for i in range(k, len(nums)):
    heappush(q, (-nums[i], i))
    while q[0][1] <= i - k: #将过时的都删掉 # 这里不是if 不然会漏删
        heappop(q)
    ans.append(-q[0][0])
return ans
```

补充知识 怎么手写一个堆

```

def maxheap(i):
    l = 2*i+1
    r = 2*i+2
    ma = i
    if l < heapsize and nums[l]>nums[i]:
        ma = l
    if r < heapsize and nums[r]>nums[ma]:
        ma = r
    if ma != i:
        nums[i],nums[ma] = nums[ma],nums[i]
        maxheap(ma)
def buildheapify():
    for i in range(heapsize//2-1,-1,-1):
        maxheap(i)

```

12. 最小覆盖子串###

给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 `s` 中的 **最短窗口子串**，使得该子串包含 `t` 中的每一个字符（包括重复字符）。如果没有这样的子串，返回空字符串 `""`。

输入: `s = "ADOBECODEBANC", t = "ABC"`

输出: `"BANC"`

解释: 最小覆盖子串 `"BANC"` 包含来自字符串 `t` 的 `'A'`、`'B'` 和 `'C'`。

关键在于怎么记录`t`里面的值 不仅记录个数 还记录了不重复的数 便于检查是否`s`子串已经具备`t`

```

def minwindow(self, s: str, t: str) -> str:
    dic = collections.defaultdict(int)
    diff = 0
    for i in t:
        if dic[i] == 0:
            diff -= 1
        dic[i] -= 1
    left = right = 0
    ansleft = ansright = 0
    minlen = len(s)
    while right < len(s):
        while right < len(s) and diff != 0:
            if dic[s[right]] == -1:
                diff += 1
            dic[s[right]] += 1
            right += 1
        while diff == 0:
            if right - left <= minlen:
                ansleft = left
                ansright = right
                minlen = right - left
            if dic[s[left]] == 0: # 只有t中的字母才会使dic[]==0 其他一开始就是0一直加
                # 会大于0
                diff -= 1 # 当左边的dic为0时 意味着现在要删除的左边界是t中的值 且删
                # 了之后字符串中将没有这个字符
                dic[s[left]] -= 1
                left += 1
    return s[ansleft:ansright]

```

13. 最大子数组和

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。**子数组**是数组中的一个连续部分。

输入: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
输出: 6
解释: 连续子数组 `[4,-1,2,1]` 的和最大，为 6。

dp数组其实只保留最后一个就行了

```
def maxSubArray(self, nums: List[int]) -> int:
    n = len(nums)
    dp = [0]*n
    start = count = temp = end = 0
    dp[0] = nums[0]
    ans = dp[0]
    for i in range(1, n):
        if nums[i] > nums[i] + dp[i-1]:
            dp[i] = nums[i]
            temp = i
            count = 0
        else:
            dp[i] = nums[i] + dp[i-1]
            count += 1
        if ans < dp[i]:
            ans = dp[i]
            start = temp
            end = start + count
    print(nums[start:end+1])
    return ans
```

14. 合并区间

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

输入: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
输出: `[[1,6],[8,10],[15,18]]`
解释: 区间 `[1,3]` 和 `[2,6]` 重叠，将它们合并为 `[1,6]`。

先排个序就简单了

```
def merge(self, intervals: List[List[int]]) -> List[List[int]]:
    n = len(intervals)
    intervals.sort(key=lambda x: x[0])# 里面的可以不写 学习一下lambda 匿名函数 前面
    # 参数后面返回值 默认它就是按第一个值排 第一个一样按第二个排
    ans = [intervals[0]]
    for i in range(1,n):
        if intervals[i][0] > ans[-1][1]:
            ans.append(intervals[i])
        elif intervals[i][0] == ans[-1][1]:
            ans[-1][1] = intervals[i][1]
        else:
            ans[-1][1] = max(ans[-1][1],intervals[i][1])
    return ans
```

15. 轮转数组

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

输入: `nums = [1,2,3,4,5,6,7]`, `k = 3`

输出: `[5,6,7,1,2,3,4]`

解释:

向右轮转 1 步: `[7,1,2,3,4,5,6]`

向右轮转 2 步: `[6,7,1,2,3,4,5]`

向右轮转 3 步: `[5,6,7,1,2,3,4]`

易错点 `k` 有可能比 `length` 长 需要取个余

```
def rotate(self, nums: List[int], k: int) -> None:
    n = len(nums)
    t = n-k%n
    for i in range(t//2):
        nums[i],nums[t-i-1] = nums[t-i-1], nums[i]
    for i in range(t,(t+n)//2):
        nums[i],nums[t+n-i-1] = nums[t+n-i-1], nums[i]
    nums.reverse()
```

16. 除自身以外数组的乘积

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积。

题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。

请 **不要使用除法**，且在 $O(n)$ 时间复杂度内完成此题。

输入: `nums = [1,2,3,4]`

输出: `[24,12,8,6]`

使用了 n 的空间 一个记录左边的积 一个记录右边的积

```
def productExceptSelf(self, nums: List[int]) -> List[int]:
    n = len(nums)
    left = [1]*(n+1)
    right = [1]*(n+1)
    ans = []
    for i in range(n):
        left[i+1]=nums[i]*left[i]
    for j in range(n-1,-1,-1):
        right[j] = nums[j]*right[j+1]
    for i in range(n):
        ans.append(left[i]*right[i+1])
    return ans
```

改进：使用O(1)的空间

第一轮 ans中的每个值为当前左面的乘积；第二轮 ans中每个左面的积和右面的积相乘

```
def productExceptSelf(self, nums: List[int]) -> List[int]:
    n = len(nums)
    ans = [0]*(n)
    l = r = 1
    ans[0] = 1
    for i in range(1,n):
        l*=nums[i-1]
        ans[i] = l
    for j in range(n-1,-1,-1):
        ans[j] = r*ans[j]
        r *= nums[j]
    return ans
```

17. 缺失的第一个正数 重点

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。

请你实现时间复杂度为 $O(n)$ 并且只使用常数级别额外空间的解决方案。

输入: `nums = [1,2,0]`

输出: 3

解释: 范围 `[1,2]` 中的数字都在数组中。

数组中出现小于0和大于数组长度的数 就一定会出现缺失 那么缺失的数一定在 $1, n$ 的闭区间里 没出现就是答案就是 $n+1$

使用hash表不符合空间复杂度

--使用原本的数组改造为hash

```
def firstMissingPositive(self, nums: List[int]) -> int:
    n = len(nums)
    # 先把<=0的置为n+1 后面计算不予考虑
    for i in range(n):
        if nums[i]<=0:
            nums[i] = n+1
    # 核心在第二个循环
    for i in range(n):
        num = abs(nums[i]) # 取回原来的值
        if num<=n: # 大于n的不考虑
```

```

        nums[num-1] = -abs(nums[num-1]) # 把值当作索引的地方的值置为- 标记已经
出现
    for i in range(n):
        if nums[i] > 0: # 表明值i+1没有出现过
            return i + 1
    return n + 1 # [1,2,3,...,n]的情况都出现过了 返回n+1

```

18. 矩阵置0

空间复杂度为m+n的好写 搞两个列表分别存行和列要改的地方

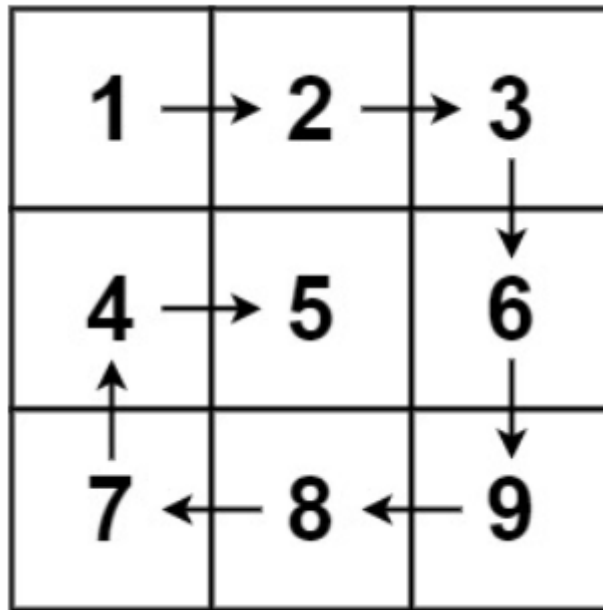
```

def setZeroes(self, matrix: List[List[int]]) -> None:
    m = len(matrix)
    n = len(matrix[0])
    flag_hang = False
    flag_lie = False
    for i in range(n): #找行中原本是否有0
        if matrix[0][i] == 0:
            flag_hang = True
    for i in range(m): #找列中原本是否有0
        if matrix[i][0] == 0:
            flag_lie = True
    for i in range(1,m):
        for j in range(1,n):
            if matrix[i][j]==0: #将对应的行列标记位置置0
                matrix[0][j]=matrix[i][0]=0
    for i in range(1,m):
        for j in range(1,n):
            if matrix[i][0] == 0 or matrix[0][j]==0: # 利用这些信息将合适的位置置0
                matrix[i][j]=0
    if flag_hang:
        for i in range(n):
            matrix[0][i] = 0
    if flag_lie:
        for i in range(m):
            matrix[i][0] = 0

```

19. 螺旋矩阵

给你一个 m 行 n 列的矩阵 `matrix`，请按照 **顺时针螺旋顺序**，返回矩阵中的所有元素。



输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]

输出: [1,2,3,6,9,8,7,4,5]

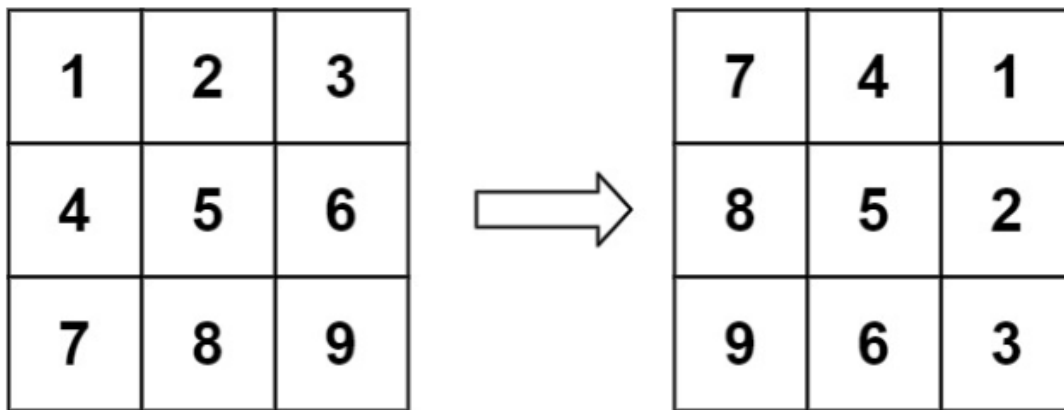
纯模拟 记得搞个层数的变量 把最外层考虑清楚就行

```
def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
    m = len(matrix)
    n = len(matrix[0])
    cengshu = 0
    ans = []
    while len(ans) < m*n:
        for i in range(cengshu, n-cengshu):
            ans.append(matrix[cengshu][i])
        for j in range(cengshu+1, m-cengshu):
            ans.append(matrix[j][n-cengshu-1])
        if m-cengshu-1 != cengshu:
            for i in range(n-1-cengshu-1, cengshu-1, -1):
                ans.append(matrix[m-cengshu-1][i])
        if n-cengshu-1 != cengshu:
            for j in range(m-1-cengshu-1, cengshu, -1):
                ans.append(matrix[j][cengshu])
        cengshu+=1
    return ans
```

20. 旋转图像

给定一个 $n \times n$ 的二维矩阵 matrix 表示一个图像。请你将图像顺时针旋转 90 度。

你必须在 原地 旋转图像，这意味着你需要直接修改输入的二维矩阵。请**不要** 使用另一个矩阵来旋转图像。



先行列对调 再左右列对调

```
def rotate(self, matrix: List[List[int]]) -> None:
    n = len(matrix)
    for i in range(n):
        for j in range(i+1):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    print(matrix)
    for i in range(n):
        for j in range(n//2):
            matrix[i][j], matrix[i][n-1-j] = matrix[i][n-1-j], matrix[i][j]
```

21. 搜索二维矩阵II

编写一个高效的算法来搜索 $m \times n$ 矩阵 matrix 中的一个目标值 target`。该矩阵具有以下特性：

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

输入: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
输出: true

从左下角开始

```
def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
    m = len(matrix)
    n = len(matrix[0])
    i = m-1
    j = 0
    while i >= 0 and j <= n-1:
        if matrix[i][j] == target:
            return True
        elif matrix[i][j] > target:
            i -= 1
        else:
            j += 1
    return False
```

22. 相交链表--简单

23. 反转链表--基础

给你单链表的头节点 `head`，请你反转链表，并返回反转后的链表。

输入: head = [1,2,3,4,5]
输出: [5,4,3,2,1]

不需要哨兵节点 记录好pre和nex就好

```
def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
    t = head
    if t is None:
        return None
    pre = None
    while t:
        nex = t.next
        t.next = pre
        pre = t
        t = nex
    return pre
```

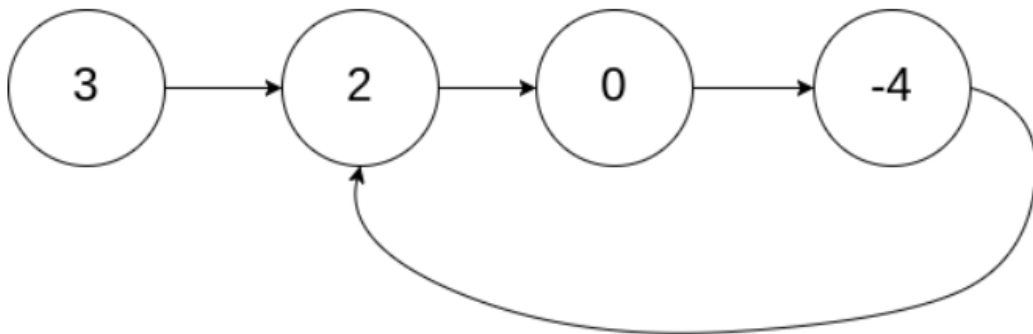
24. 回文链表

翻转后面部分的链表

快慢指针确定中间位置或者就遍历确定长度

反转后面的与前面的比较

25. 环形链表



输入: head = [3,2,0,-4], pos = 1
输出: true
解释: 链表中有一个环, 其尾部连接到第二个节点。

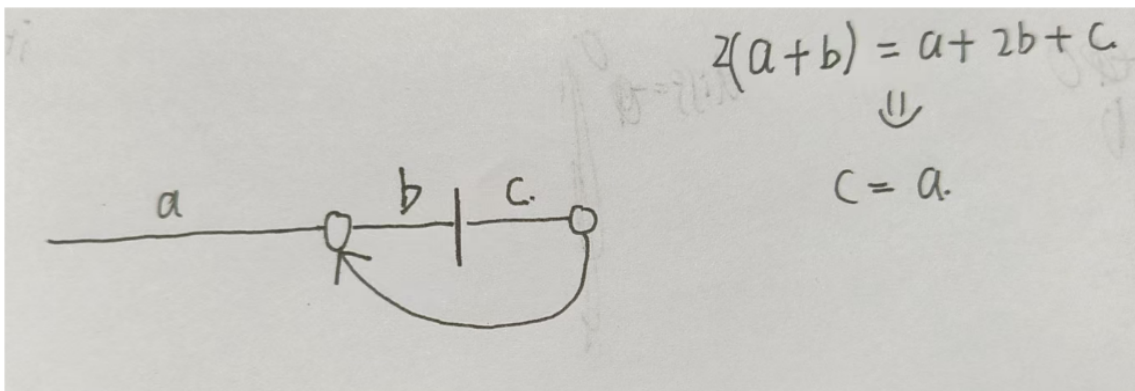
快慢指针

```
def hasCycle(self, head: Optional[ListNode]) -> bool:
    low = fast = head
    while fast and fast.next:
        low = low.next
        fast = fast.next.next
        if low == fast:
            return True
    return False
```

26. 环形链表 II

快慢指针

证明: 假设一圈就追上了



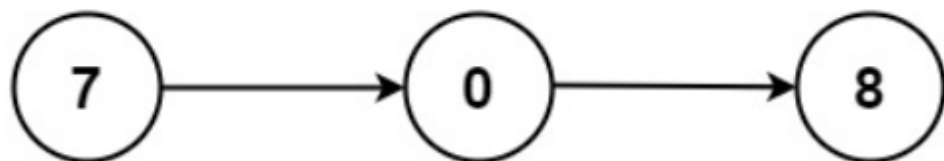
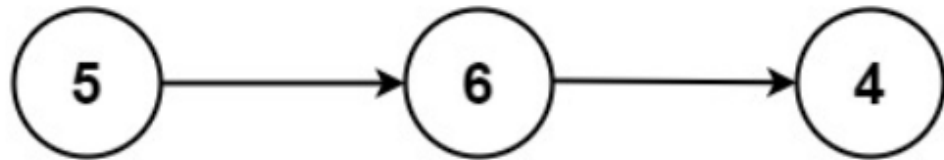
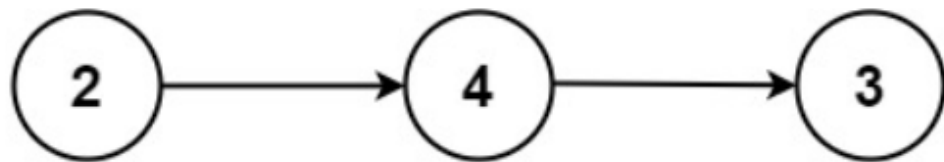
```
def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
    low = fast = head
    t = False
    while fast and fast.next:
        low = low.next
        fast = fast.next.next
        if low == fast:
            t = True
            break
    if t==False:
        return None
    else:
        fast = head
        while fast!=low:
            low = low.next
            fast=fast.next
    return fast
```

27. 合并两个有序链表

加个哨兵

```
def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode])
-> Optional[ListNode]:
    hair = ListNode(0)
    t = hair
    l1 = list1
    l2 = list2
    while l1 and l2:
        if l1.val <= l2.val:
            t.next = l1
            t = t.next
            l1 = l1.next
        else:
            t.next = l2
            t = t.next
            l2 = l2.next
    if l1:
        t.next = l1
    if l2:
        t.next = l2
    return hair.next
```

28. 两数相加



模拟 注意进位的问题

29. 删除链表的第n个节点

快慢指针 快指针先提前走n+1次得到删的节点的前一个

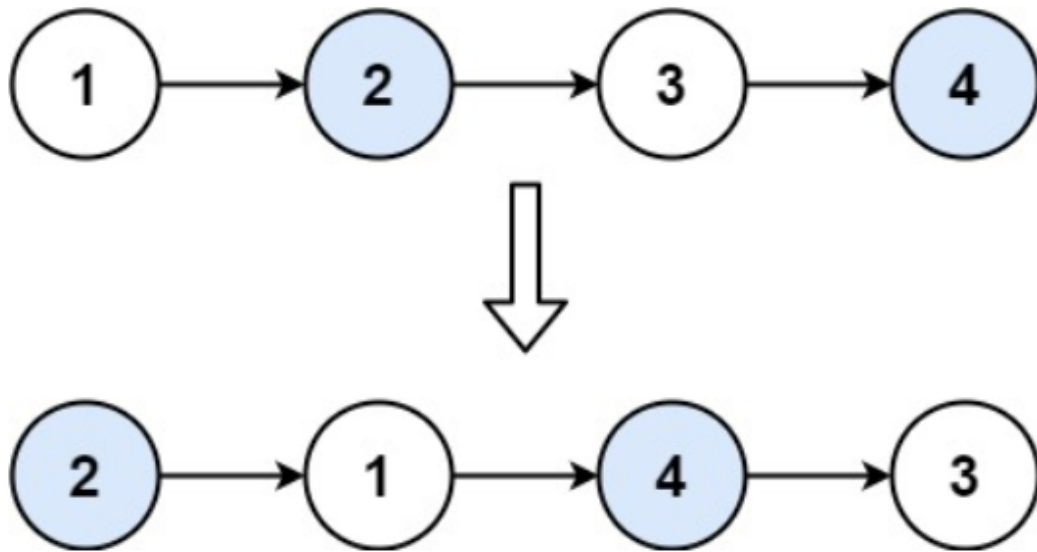
尤其需要注意的是 只有一个删第一个

此时就体现处哨兵的作用

```

def removeNthFromEnd(self, head: Optional[ListNode], n: int) ->
Optional[ListNode]:
    hair = ListNode(0)
    hair.next = head
    low = fast = hair
    for i in range(n+1):
        fast = fast.next
    while fast:
        fast=fast.next
        low = low.next
    pre = low
    low = low.next
    nex = low.next
    pre.next = nex
    return hair.next
  
```

30. 两两交换链表中的节点



```

def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
    hair = ListNode(0)
    hair.next = head
    t = hair
    nex = None
    while t.next and t.next.next:
        l1 = t.next
        l2 = t.next.next
        nex = l2.next
        t.next = l2
        l2.next = l1
        l1.next = nex
        t = l1
    return hair.next

```

31. K个一组翻转链表

上一个题的强化版

```

def reverseKGroup(self, head: Optional[ListNode], k: int) ->
Optional[ListNode]:
    def fanzhuan(start, end):
        pre = None
        t = start
        endnex = end.next
        while t != endnex:
            nex = t.next
            t.next = pre
            pre = t
            t = nex
        return end, start
    hair = ListNode(0)
    hair.next = head
    end = hair
    pre = end
    while True:
        for i in range(k):
            if end.next is None: # 如果剩下的不够了 直接返回结果
                return hair.next

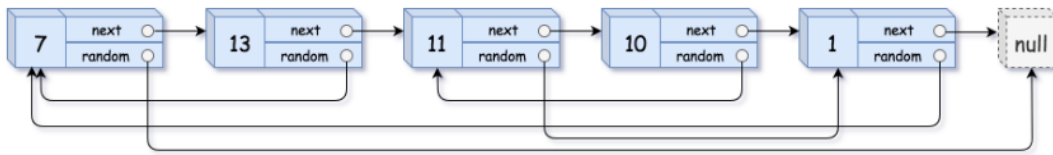
```

```

        end = end.next
    start = pre.next
    nex = end.next
    start,end = fanzhuang(start,end)
    end.next = nex
    pre.next = start
    pre = end

```

32. 随机链表的复制



```

"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""

```

法一借助字典 key为原始链表的节点 value为新建的链表节点

```

def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
    dic = {} # 原始链表和新链表构建的一对一关系 也是后面优化空间的出发点
    # key是原链表节点 value是新建链表节点
    t = head
    if head is None:
        return None
    while t:
        dic[t] = Node(t.val)
        t = t.next
    t = head
    while t:
        tt = dic[t]
        if t.next:
            tt.next = dic[t.next]
        if t.random:
            tt.random = dic[t.random]
        t = t.next
    return dic[head]

```

法二 无额外空间

之前的法一中使用旧指向新节点的字典

现在我们真实的构建一个旧节点指向一个新节点

尤其最后拆出来时要注意 不要破坏原链表结构

```

def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
    if head == None:

```

```

        return head
    t = head
    while t:
        tt = t.next
        t.next = Node(t.val, tt)
        t = tt
    t = head
    while t:
        tt = t.next
        if t.random:
            tt.random = t.random.next
        t = t.next.next
    newhead = head.next
    t = newhead
    t_old = head
    while t.next:
        nex = t.next
        t.next = t.next.next
        t = t.next
        t_old.next = nex
        t_old = t_old.next
    t_old.next = None
    return newhead

```

33. 排序链表 重点

使用自底向上代码比较冗长

```

def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
    def merge(head1: ListNode, head2: ListNode) -> ListNode:
        ...

    if not head:
        return head
    length = 0
    node = head
    while node:
        length += 1
        node = node.next
    dummyHead = ListNode(0, head)
    subLength = 1
    while subLength < length:
        prev, curr = dummyHead, dummyHead.next
        while curr:
            head1 = curr
            for i in range(1, subLength):
                if curr.next:
                    curr = curr.next
                else:
                    break
            head2 = curr.next
            curr.next = None
            curr = head2
            for i in range(1, subLength):
                if curr:
                    curr = curr.next

```

```

        else:
            break
    nex = None # 这里是为了将这两组与之后的断开 记录后面的起点
    if curr:
        nex = curr.next
        curr.next = None
    merged = merge(head1, head2)
    prev.next = merged #与前面重新链接
    while prev.next:
        prev = prev.next
    curr = nex
    subLength*=2
    return dummyHead.next

```

34. 合并k个升序链表

保证两个元素间可以比较 对该类设置__lt__

还需要注意heapify等函数参数是list类型

```

ListNode.__lt__ = lambda a, b: a.val < b.val
def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
    dui = []
    for i in lists:
        if i:
            dui.append(i)
    heapify(dui)
    hair = ListNode(0)
    t = hair
    while dui:
        h = heappop(dui)
        if h.next:
            heappush(dui, h.next)
        t.next = h
        t = t.next
    return hair.next

```

35. LRU缓存

请你设计并实现一个满足 [LRU \(最近最少使用\) 缓存](#) 约束的数据结构。

实现 `LRUCache` 类:

- `LRUCache(int capacity)` 以 **正整数** 作为容量 `capacity` 初始化 LRU 缓存
- `int get(int key)` 如果关键字 `key` 存在于缓存中，则返回关键字的值，否则返回 `-1`。
- `void put(int key, int value)` 如果关键字 `key` 已经存在，则变更其数据值 `value`；如果不存在，则向缓存中插入该组 `key-value`。如果插入操作导致关键字数量超过 `capacity`，则应该 **逐出** 最久未使用的关键字。

函数 `get` 和 `put` 必须以 $O(1)$ 的平均时间复杂度运行。

输入

```

["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

```

输出

```

[null, null, null, 1, null, -1, null, -1, 3, 4]

```

解释

```

LRUCache lruCache = new LRUCache(2);
lruCache.put(1, 1); // 缓存是 {1=1}
lruCache.put(2, 2); // 缓存是 {1=1, 2=2}
lruCache.get(1);    // 返回 1
lruCache.put(3, 3); // 该操作会使得关键字 2 作废, 缓存是 {1=1, 3=3}
lruCache.get(2);    // 返回 -1 (未找到)
lruCache.put(4, 4); // 该操作会使得关键字 1 作废, 缓存是 {4=4, 3=3}
lruCache.get(1);    // 返回 -1 (未找到)
lruCache.get(3);    // 返回 3
lruCache.get(4);    // 返回 4

```

核心 定义一个双指针类

```

class Dnode:
    def __init__(self, key = 0, val=0, pre=None, nex=None):
        self.key = key
        self.val = val
        self.pre = pre
        self.nex = nex

```

定义 一个头指针和尾指针作为哨兵 方便

```

def __init__(self, capacity: int):
    self.capacity = capacity
    self.num = 0
    self.dic = {}
    self.head = Dnode(-1)
    self.tail = Dnode(-1)
    self.head.nex = self.tail
    self.tail.pre = self.head

```

需要注意易错点:

新插入一个时 先判断是不是原先有的 有的话需要改掉值 并且放在最后; 如果不存在还要判断是否缓存达到上限 达到需要先抛出头部 然后再尾插

新输出一个时 先判断有没有 没有直接返回-1 有则输出对应的值 并且将其放在最后

36. 二叉树中序遍历 **重点** XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

要学会递归 非递归 Morris

```

def inorder(self, root, ans):
    if not root:
        return
    self.inorder(root.left, ans)
    ans.append(root.val)
    self.inorder(root.right, ans)
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    # 递归
    ans = []
    self.inorder(root, ans)
    return ans

```

```
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    sstack = []
    while root or sstack:
        while root:
            sstack.append(root)
            root = root.left
        root = sstack.pop()
        ans.append(root.val)
        root = root.right
    return ans
```

```
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    while root:
        if root.left:
            # 如果left存在 先找到该节点的最右节点 也就是中序时的上一个节点
            pre = root.left
            while pre.right and pre.right!=root:
                pre =pre.right
            if pre.right == None:#如果空 表明第一次遇到 先给它指向root
                pre.right = root
                root =root.left
            else:#第二次遇到 表明之前已经访问完了 取消链接 并加入root的值
                ans.append(root.val)
                pre.right = None
                root = root.right
        else:
            ans.append(root.val)
            root= root.right
    return ans
```

补充: 前序

```
def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    sstack = []
    while root or sstack:
        while root:
            ans.append(root.val)
            sstack.append(root)
            root = root.left
        root = sstack.pop()
        root = root.right
    return ans
```

```
def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    while root:
        if root.left:
            pre = root.left
            while pre.right and pre.right != root:
                pre = pre.right
```

```

        if pre.right is None:
            pre.right = root
            ans.append(root.val)
            root=root.left
        else:
            root=root.right
            pre.right = None
    else:
        ans.append(root.val)
        root=root.right
return ans

```

后序

```

def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    sstack = []
    pre = None
    while root or sstack:
        while root:
            sstack.append(root)
            root=root.left
        root = sstack.pop()
        if root.right is None or root.right==pre:
            ans.append(root.val)
            pre = root
            root = None
        else:
            sstack.append(root)
            root = root.right
    return ans

```

37. 二叉树的最大深度

递归

```

def maxDepth(self, root: Optional[TreeNode]) -> int:
    if root is None:
        return 0
    return 1+max(self.maxDepth(root.left),self.maxDepth(root.right))

```

38. 翻转二叉树

递归

```

def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
    if root is None:
        return None
    l = self.invertTree(root.left)
    r = self.invertTree(root.right)
    t = l
    root.left = r
    root.right = t
    return root

```

39. 对称二叉树

```
def issymmetric(self, root: Optional[TreeNode]) -> bool:
    def isTrue(root1, root2):
        if root1 and root2:
            if root1.val == root2.val:
                return isTrue(root1.left, root2.right) and
isTrue(root1.right, root2.left)
            else:
                return False
        elif root1 is None and root2 is None:
            return True
        else:
            return False
    return isTrue(root.left, root.right)
```

40. 二叉树的直径

给你一棵二叉树的根节点，返回该树的 **直径**。

二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度**。这条路径可能经过也可能不经过根节点 `root`。

两节点之间路径的 **长度** 由它们之间边数表示。

不可以直接以根节点算左边深度和右边深度 这样子会漏掉一些情况

还是要按求深度的思想 只是再求深度的时候就动态地修改ans值 这样子就不会只考虑根节点了

```
class Solution:
    ans = 0
    def depth(self, root):
        if root is None:
            return 0
        left = self.depth(root.left)
        right = self.depth(root.right)
        self.ans = max(left+right, self.ans)
        return 1+max(left, right)
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        self.depth(root)
        return self.ans
```

41. 二叉树的层序遍历

队列的使用 搞一个count来记录每一层的个数

42. 将有序数组转化为二叉搜索树

二分着建立树 一定是平衡的

```
def erfen(self, nums, l, r):
    if l <= r:
        mid = (l+r)//2
        root = TreeNode(nums[mid])
        root.left = self.erfen(nums, l, mid-1)
        root.right = self.erfen(nums, mid+1, r)
        return root
    return None

def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
    return self.erfen(nums, 0, len(nums)-1)
```

43. 验证二叉搜索树###

两种方法 笨办法 使用中序遍历 检查中序的结果是否递增

第二种 使用递归递归的判断左面和右面

判断左面时 根节点是最大值 判断右面时根节点是最小值

```
def isValidBST(self, root: Optional[TreeNode]) -> bool:
    def panduan(node, lower, upper):
        if node is None:
            return True
        val = node.val
        if val <= lower or val >= upper:
            return False
        return panduan(node.left, lower, val) and panduan(node.right, val, upper)
    return panduan(root, float("-inf"), float("inf"))
```

44. 二叉搜索树中第K小的元素

中序遍历到第k个即可

```
def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
    sstack = []
    while root or sstack:
        while root:
            sstack.append(root)
            root = root.left
        root = sstack.pop()
        k -= 1
        if k == 0:
            return root.val
        root = root.right
```

45. 二叉树的右视图

层序遍历每层取最后一个就行

46. 二叉树展开为链表

给你二叉树的根结点 `root`，请你将它展开为一个单链表：

- 展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。
- 展开后的单链表应该与二叉树 [先序遍历](#) 顺序相同。

使用类似morris的方法 找到当前节点的最右节点（即先序遍历的最后一个）和左节点

```

def flatten(self, root: Optional[TreeNode]) -> None:
    t = root
    while t:
        if t.left:
            newright = t.left
            pre = newright
            while pre.right:
                pre = pre.right
            nex = t.right
            t.right = newright
            pre.right = nex
            t.left = None
        t = t.right

```

47. 从前序和中序遍历构造二叉树 重点 XX

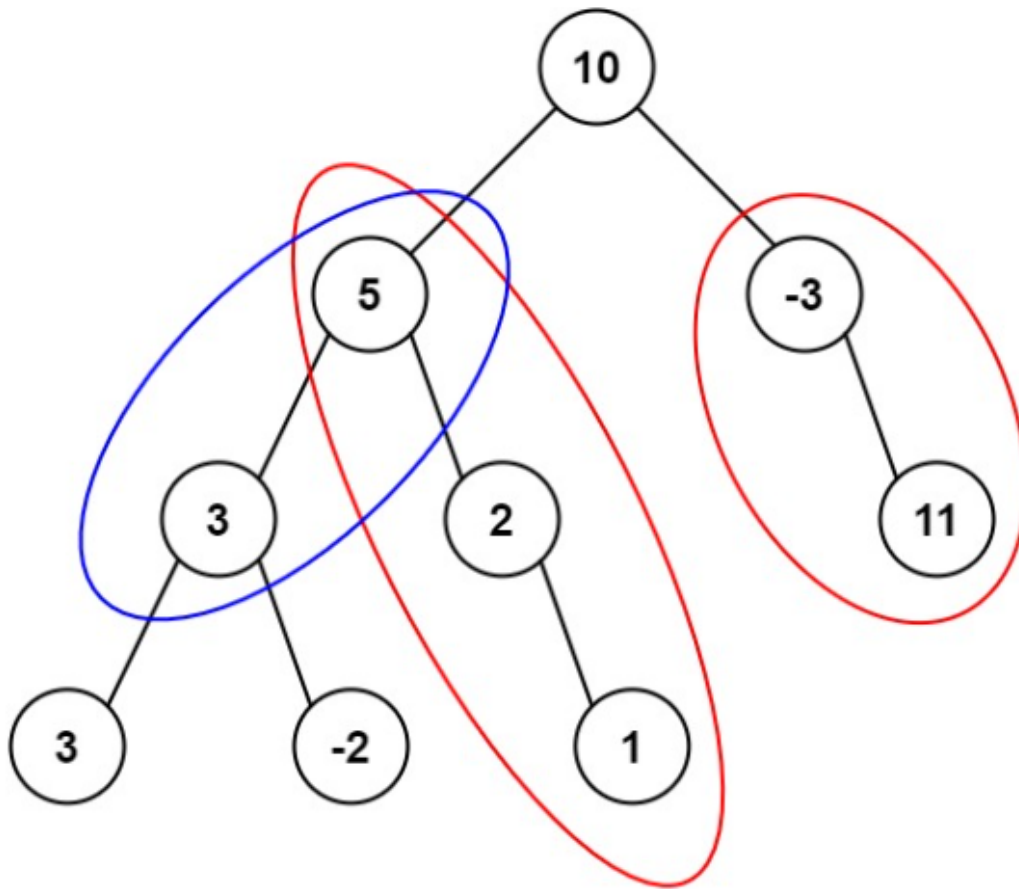
无重复元素

```

def buildTree(self, preorder: List[int], inorder: List[int]) ->
Optional[TreeNode]:
    dic = {}
    n = len(inorder)
    for i in range(n):
        dic[inorder[i]] = i # 以中序的构建dic 快速根据前序的第一个值找到在中序中的位置
    def goujian(preleft, preright, inleft, inright):
        if preleft > preright:
            return None
        value = preorder[preleft]
        root = TreeNode(value)
        inorder_index = dic[value] # 只有中序的索引才能确定左子树和右子树有多少个
        size_left = inorder_index - inleft
        root.left = goujian(preleft+1, preleft+size_left, inleft, inorder_index-
1)
        root.right =
goujian(preleft+size_left+1, preright, inorder_index+1, inright)
        return root
    return goujian(0, n-1, 0, n-1)

```

48. 路径总和III* XX



回溯思想

假设当前从根节点 $root$ 到节点 $node$ 的前缀和为 $curr$ ，则此时我们在已保存的前缀和查找是否存在前缀和刚好等于 $curr - targetSum$ 。假设从根节点 $root$ 到节点 $node$ 的路径中存在节点 pi 到根节点 $root$ 的前缀和为 $curr - targetSum$ ，则节点 $pi+1$ 到 $node$ 的路径上所有节点的和一定为 $targetSum$ 。

```
def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
    prefix = collections.defaultdict(int)
    prefix[0] = 1
    def dfs(root, curr):
        if root is None:
            return 0
        curr += root.val
        ret = 0
        ret += prefix[curr - targetSum]
        prefix[curr] += 1
        ret += dfs(root.left, curr)
        ret += dfs(root.right, curr)
        prefix[curr] -= 1
        return ret
    return dfs(root, 0)
```

49. 二叉树的最近公共祖先

思路：深搜标记每个节点的父节点--将p的父节点都标记好--遍历q的父节点 第一个为真的就是最近祖先

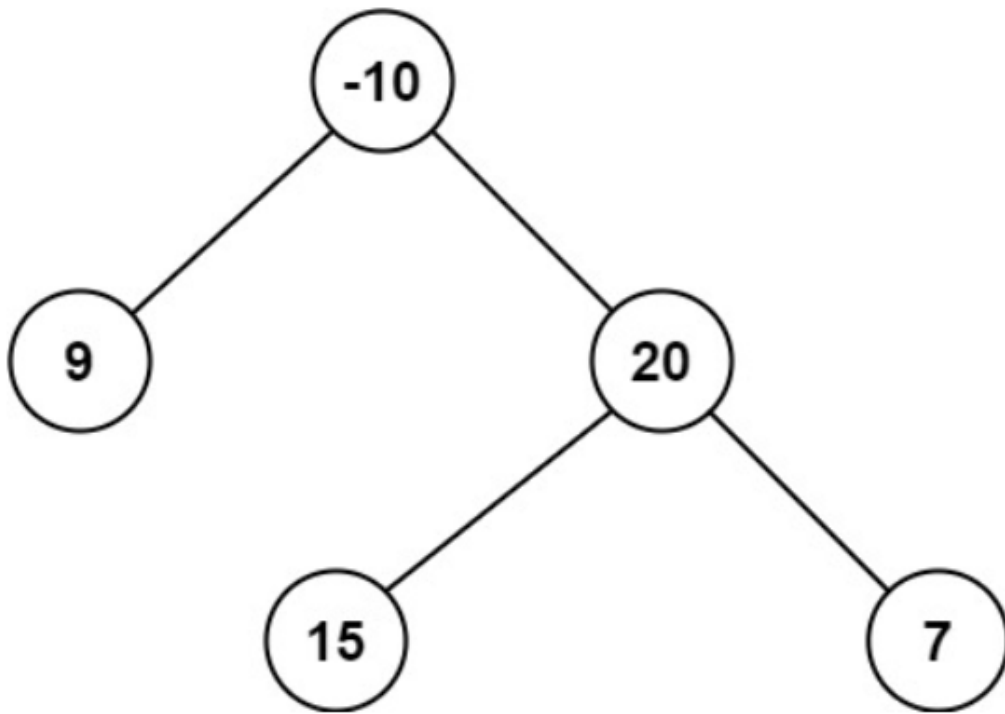
```
def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q:
    'TreeNode') -> 'TreeNode':
    fa = {}
```

```

vis = {}
def dfs(root):
    if root.left:
        fa[root.left.val] = root
        dfs(root.left)
    if root.right:
        fa[root.right.val] = root
        dfs(root.right)
    fa[root.val] = None
    dfs(root)
while p:
    vis[p.val] = True
    p = fa[p.val]
while q:
    if vis.get(q.val, False):
        return q
    q = fa[q.val]
return None

```

50. 二叉树的最大路径和



输入: root = [-10,9,20,null,null,15,7]

输出: 42

解释: 最优路径是 15 -> 20 -> 7 , 路径和为 15 + 20 + 7 = 42

与**二叉树的直径**代码结构比较像 那个是算左右深度 这个是算左右最值 返回的时候都是返回一边的最值

```

ans = float("-inf")
def maxPathSum(self, root: Optional[TreeNode]) -> int:
    def dfs(root):
        if root is None:
            return 0
        left = max(0,dfs(root.left))
        right = max(0,dfs(root.right))
        self.ans = max(self.ans, root.val+left+right)
        return root.val+max(left,right)
    dfs(root)
    return self.ans

```

51. 岛屿数量

给你一个由 '1' (陆地) 和 '0' (水) 组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

```

输入: grid = [
  ['1','1','1','1','0'],
  ['1','1','0','1','0'],
  ['1','1','0','0','0'],
  ['0','0','0','0','0']
]
输出: 1

```

如果不让改原数组 可以加一个vis来标记是否访问 等同于赋值0

```

def numIslands(self, grid: List[List[str]]) -> int:
    ans = 0
    m = len(grid)
    n = len(grid[0])
    vis = [[False]*n for _ in range(m)]
    def yandiao(i,j):
        for x,y in [(i+1,j),(i-1,j),(i,j+1),(i,j-1)]:
            if 0<=x<=m-1 and 0<=y<=n-1 and grid[x][y]=='1' and vis[x][y] == False:
                vis[x][y] = True
                yandiao(x,y)
    for i in range(m):
        for j in range(n):
            if grid[i][j]=='1' and vis[i][j] == False:
                yandiao(i,j)
                ans+=1
    return ans

```

52. 腐烂的橘子

bfs模拟

有些特殊情况 无法全部腐烂 和 一开始就没有新鲜橘子

队列那儿不需要专门搞个计数的 直接while que进来后先记录下长度就行

```

def orangesRotting(self, grid: List[List[int]]) -> int:

```

```

ans = 0
m = len(grid)
n = len(grid[0])
que = []
count = 0
for i in range(m):
    for j in range(n):
        if grid[i][j] != 1:
            count+=1
if count==m*n:
    return 0
def infect(i,j):
    for i,j in (i+1,j),(i-1,j),(i,j+1),(i,j-1):
        if 0<=i<=m-1 and 0<=j<=n-1 and grid[i][j]==1:
            grid[i][j] = 2
            que.append([i,j])
for i in range(m):
    for j in range(n):
        if grid[i][j] == 2:
            que.append([i,j])
while que:
    l = len(que)
    ans+=1
    for i in range(l):
        t = que.pop(0)
        infect(t[0],t[1])

for i in range(m):
    for j in range(n):
        if grid[i][j] == 1:
            return -1
return ans-1

```

53. 课程表

你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1`。

在选修某些课程之前需要一些先修课程。先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [ai, bi]`，表示如果要学习课程 `ai` 则 **必须** 先学习课程 `bi`。

- 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0`，你需要先完成课程 `1`。

请你判断是否可能完成所有课程的学习？如果可以，返回 `true`；否则，返回 `false`。

输入: `numCourses = 2, prerequisites = [[1,0]]`

输出: `true`

解释: 总共有 2 门课程。学习课程 1 之前，你需要完成课程 0。这是可能的。

模拟拓扑排序

每轮将入度为0的算到ans里并且将它对应指向的元素的入度都减1

```

def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
    dic = collections.defaultdict(int)
    for i in prerequisites:
        dic[i[0]]+=1
    ans = 0
    vis = [False]*numCourses

```

```

while True:
    count = 0
    for i in range(numCourses):
        if dic[i]==0 and vis[i]==False:
            ans+=1
            vis[i]=True
            for j in prerequisites:
                if j[1]==i:
                    dic[j[0]]-=1
            count +=1
    if count==0:
        return ans==numCourses

```

54. 实现前缀树

[Trie](#)（发音类似 "try"）或者说 **前缀树** 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补全和拼写检查。

请你实现 Trie 类：

- `Trie()` 初始化前缀树对象。
- `void insert(String word)` 向前缀树中插入字符串 `word`。
- `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true`（即，在检索之前已经插入）；否则，返回 `false`。
- `boolean startswith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix`，返回 `true`；否则，返回 `false`。

输入

```

["Trie", "insert", "search", "search", "startswith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]

```

输出

```

[null, null, true, false, true, null, true]

```

解释

```

Trie trie = new Trie();
trie.insert("apple");
trie.search("apple");    // 返回 True
trie.search("app");      // 返回 False
trie.startswith("app");  // 返回 True
trie.insert("app");
trie.search("app");      // 返回 True

```

关键步骤 init 定义一个26个字母对应的指向 指向的是Trie 记录是否为结尾

```

class Trie:
    def __init__(self):
        self.childred = [None]*26
        self.isend = False
    def insert(self, word: str) -> None:
        node = self#-----关键-----
        for ch in word:
            t = ord(ch)-ord('a')
            if not node.childred[t]:#插入时要判断 防止已经存在的被覆盖掉
                node.childred[t] = Trie()#只有为空的时候才新建
            node = node.childred[t]
        node.isend = True

```

```

def searchPrefix(self, word):#找到所查询的最后一个Trie节点
    node = self
    for ch in word:
        t = ord(ch)-ord('a')
        if not node.childred[t]:
            return None
        node = node.childred[t]
    return node
def search(self, word: str) -> bool:
    node = self.searchPrefix(word)
    return node is not None and node.isend
def startswith(self, prefix: str) -> bool:
    node = self.searchPrefix(prefix)
    return node is not None

```

55. 全排列

```

def permute(self, nums: List[int]) -> List[List[int]]:
    ans = []
    n = len(nums)
    vis = [False]*n
    t = []
    def dfs(depth):
        if depth == n:
            ans.append(t[:])# ---搞一个拷贝存入 不然存的是引用
        for i in range(n):
            if vis[i]!=True:
                vis[i]=True
                t.append(nums[i])
                dfs(depth+1)
                t.pop()
                vis[i]=False
    dfs(0)
    return ans

```

56. 子集

输入: nums = [1,2,3]

输出: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]

```

def subsets(self, nums: List[int]) -> List[List[int]]:
    n = len(nums)
    ans = []
    t = []
    def dfs(depth):
        if depth==n:
            ans.append(t[:])
            return
        t.append(nums[depth])
        dfs(depth+1)#选它下一层
        t.pop()
        dfs(depth+1)#不选它下一层
    dfs(0)
    return ans

```

57. 电话号码的组合

输入: digits = "23"

输出: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

关键在于那个字典会很方便后续的处理

```
def letterCombinations(self, digits: str) -> List[str]:
    dic = { "2": "abc",
            "3": "def",
            "4": "ghi",
            "5": "jkl",
            "6": "mno",
            "7": "pqrs",
            "8": "tuv",
            "9": "wxyz",
          }
    ans = []
    t = []
    def dfs(depth):
        if depth == len(digits):
            ans.append("".join(t[:]))
            return
        s = dic[digits[depth]]
        for ch in s:
            t.append(ch)
            dfs(depth+1)
            t.pop()
    dfs(0)
    return ans
```

58. 组合求和

输入: candidates = [2,3,6,7], target = 7

输出: [[2,2,3],[7]]

解释:

2 和 3 可以形成一组候选, $2 + 2 + 3 = 7$ 。注意 2 可以使用多次。

7 也是一个候选, $7 = 7$ 。

仅有这两种组合。

```
def combinationSum(self, candidates: List[int], target: int) ->
List[List[int]]:
    ans = []
    t = []
    n = len(candidates)
    candidates.sort() #排完序之后会很方便 dfs再加一个参数就可以解决重复的问题
    def dfs(curr, index):
        if curr == target:
            ans.append(t[:])
            return
        for i in range(index, n):
            curr += candidates[i]
            if curr > target: #及时跳出 缓解时间复杂度
```

```

        break
    t.append(candidates[i])
    dfs(curr,i)
    curr-=candidates[i]
    t.pop()
dfs(0,0)
return ans

```

59. 括号生成

输入: $n = 3$

输出: ["((()))", "(()())", "(())()", "()(())", "()()()"]

关键在于什么时候加左括号和右括号

```

def generateParenthesis(self, n: int) -> List[str]:
    ans = []
    t = []
    def dfs(left, right):
        if left+right==2*n:
            ans.append("".join(t[:]))
            return
        if left < n:
            t.append('(')
            dfs(left+1,right)
            t.pop()
        if right<left:
            t.append(')')
            dfs(left,right+1)
            t.pop()
    dfs(0,0)
    return ans

```

60. 单词搜索

A	B	C	E
S	F	C	S
A	D	E	E

输入: board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word = "ABCCED"
输出: true

起点保存好 各自深搜 注意vis及时置True和False

```
class Solution:
    ans = False
    def exist(self, board: List[List[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])
        s = word[0]
        que = []
        vis = [[False]*n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if board[i][j]==s:
                    que.append([i,j])
        def dfs(i,j,dep):
            if dep == len(word):
                self.ans = True
                return
            for x,y in (i+1,j),(i-1,j),(i,j+1),(i,j-1):
                if 0<=x<=m-1 and 0<=y<=n-1 and board[x][y]==word[dep] and vis[x][y]==False:
                    vis[x][y] = True
                    dfs(x,y,dep+1)
                    vis[x][y] = False
        while que:
            t = que.pop(0)
            vis[t[0]][t[1]]=True
            dfs(t[0],t[1],1)
            vis[t[0]][t[1]]=False
        return self.ans
```

61. 分割回文串

输入: s = "aab"
输出: [["a","a","b"],["aa","b"]]

回文dp数组的构建+回溯

```
def partition(self, s: str) -> List[List[str]]:
    n = len(s)
    dp = [[True]*n for _ in range(n)]
    for j in range(0,n):
        for i in range(0,j):
            dp[i][j] = (s[i]==s[j] and dp[i+1][j-1])
    ans = []
    t = []
    def dfs(depth):
        if depth==n:
            ans.append(t[:])
```

```

        return
    for i in range(depth,n):
        if dp[depth][i]:
            t.append(s[depth:i+1])
            dfs(i+1)
            t.pop()

    dfs(0)
    return ans

```

62. N皇后

难点 返回值那儿

```

def solvenQueens(self, n: int) -> List[List[str]]:
    ans = []
    t = [ "."*n for _ in range(n)]
    def panduan(i,j):
        x= i-1
        y= j-1
        for m in range(i):
            if t[m][j]=='Q':
                return False
        while x>=0 and y>=0:
            if t[x][y]=='Q':
                return False
            x-=1
            y-=1
        x= i-1
        y= j+1
        while x>=0 and y<n:
            if t[x][y]=='Q':
                return False
            x-=1
            y+=1
        return True
    def dfs(depth):
        if depth == n:
            zan = []
            for i in range(n):#-----关键-----
                zan.append("".join(t[i]))
            ans.append(zan[:])
            return
        for j in range(n):
            if panduan(depth,j):
                t[depth][j]='Q'
                dfs(depth+1)
                t[depth][j]='.'
    for i in range(n):
        t[0][i] = 'Q'
        dfs(1)
        t[0][i]= '.'
    return ans

```

63. 搜索插入位置

输入: nums = [1,3,5,6], target = 5
输出: 2

```
def searchInsert(self, nums: List[int], target: int) -> int:
    n = len(nums)
    left = 0
    right = n-1
    while left<=right: # 闭区间不容易错
        mid = (left+right)//2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid -1
    return left
```

64. 搜索二维矩阵

从左下角开始就行

前面还有个题是搜索二维矩阵II 一样的解法都适用

```
def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
    i = len(matrix) - 1
    j = 0
    n = len(matrix[0])
    while i>=0 and j <n:
        if matrix[i][j] == target:
            return True
        elif matrix[i][j] > target:
            i-=1
        else:
            j+=1
    return False
```

65. 在排序数组中找第一个和第二个

输入: nums = [5,7,7,8,8,10], target = 8
输出: [3,4]

```
def searchRange(self, nums: List[int], target: int) -> List[int]:
    # 思路就是二分找左面和右面
    ans = [-1,-1]
    n = len(nums)
    left = 0
    right = n-1
    while left<=right:
        mid = (left+right)//2
        if nums[mid] >= target:
            right = mid-1
            if nums[mid] == target:
                ans[0]=mid
        else:
            left = mid +1
```

```

if ans[0]==-1:
    return ans
left = 0
right = n-1
while left<=right:
    mid = (left+right)//2
    if nums[mid] > target:
        right = mid-1
    else:
        if nums[mid] == target:
            ans[1]=mid
            left = mid +1
return ans

```

66. 搜索旋转排序数组

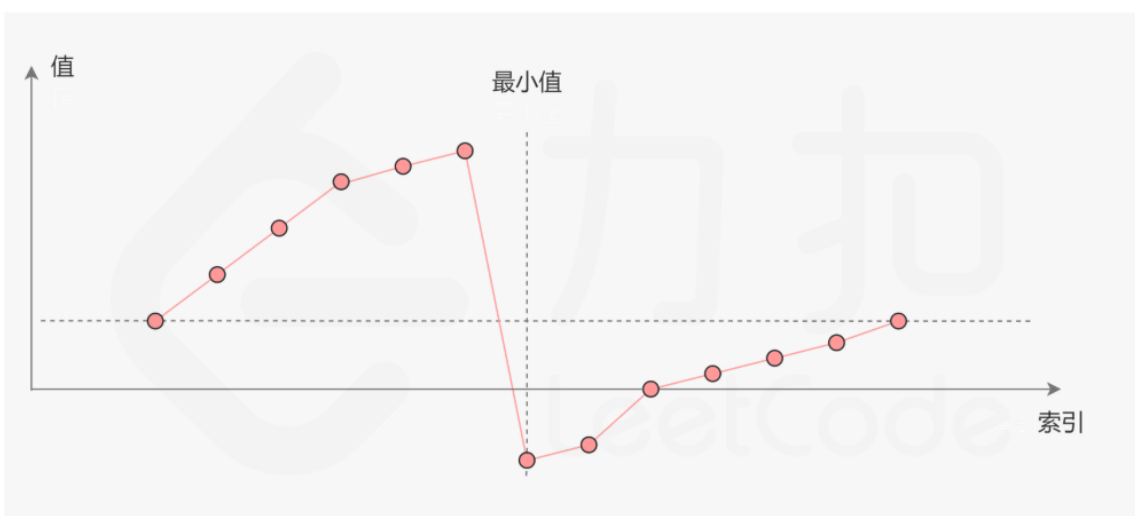
输入: nums = [4,5,6,7,0,1,2], target = 0
输出: 4

```

def search(self, nums: List[int], target: int) -> int:
    n = len(nums)
    left = 0
    right = n-1
    ans = -1
    while left<=right:
        mid = (left+right)//2
        if target == nums[mid]:
            return mid
        if nums[0] < nums[mid]:
            if nums[0] <= target < nums[mid]:
                right = mid -1
            else:
                left = mid+1
        else:#这里是else 不是if。。 防止数组只有一位陷入死循环
            if nums[mid]<target<= nums[n-1]:
                left = mid+1
            else:
                right = mid -1
    return ans

```

67. 寻找旋转排序矩阵的最小值



```
def findMin(self, nums: List[int]) -> int:
    ans = 5001
    left = 0
    n = len(nums)
    right = n-1
    while left<=right:
        mid = (left+right)//2
        ans = min(ans,nums[mid])
        if nums[0] <= nums[mid]:
            left = mid +1
        else:
            right = mid
    return ans
```

68. 寻找两个正序数组的中位数 重点

输入: nums1 = [1,3], nums2 = [2]
 输出: 2.00000
 解释: 合并数组 = [1,2,3] , 中位数 2

关键在于编写一个找两个数组中的第k个

可以每次后移 $k//2-1$ 记得要减一 不然会多移动几位导致漏掉结果

```
def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
    m = len(nums1)
    n = len(nums2)
    total = m+n
    def getK(k):
        index1 = index2 = 0
        while True:
            if index1==m:
                return nums2[index2+k-1]
            if index2==n:
                return nums1[index1+k-1]
            if k == 1:
                return min(nums1[index1],nums2[index2])
            newIndex1 = min(m-1, index1 + k//2 -1)
            newIndex2 = min(n-1, index2 + k//2 -1)
            if nums1[newIndex1] <= nums2[newIndex2]:
                k -= newIndex1-index1+1
                index1 = newIndex1 + 1
            else:
                k -= newIndex2-index2+1
                index2 = newIndex2 + 1

    if total%2==1:
        return getK(total//2+1)
    else:
        return (getK(total//2)+getK(total//2+1))/2
```

69. 有效的括号

输入: s = "()[]{}"

输出: true

输入: s = "[]"

输出: false

```
def isValid(self, s: str) -> bool:
    sstack = []
    for i in s:
        if i == '(' or i == '[' or i == '{':
            sstack.append(i)
        else:
            if sstack:
                t = sstack.pop()
                if i == ')' and t != '(' or i == ']' and t != '[' or i == '}' and t != '{':
                    return False
            else:
                return False
    if sstack:
        return False
    else:
        return True
```

70. 最小栈

设计一个支持 `push` , `pop` , `top` 操作, 并能在常数时间内检索到最小元素的栈。

实现 `MinStack` 类:

- `MinStack()` 初始化堆栈对象。
- `void push(int val)` 将元素`val`推入堆栈。
- `void pop()` 删除堆栈顶部的元素。
- `int top()` 获取堆栈顶部的元素。
- `int getMin()` 获取堆栈中的最小元素。

```
class MinStack:
    def __init__(self):
        self.minstack = [math.inf] # 这个栈每次记录的都是当前值进来后对应的最小值
        self.ans = []

    def push(self, val: int) -> None:
        self.ans.append(val)
        self.minstack.append(min(self.minstack[-1], val))

    def pop(self) -> None:
        self.minstack.pop()
        self.ans.pop()

    def top(self) -> int:
        return self.ans[-1]

    def getMin(self) -> int:
        return self.minstack[-1]
```

71. 字符串解码

输入: `s = "3[a]2[bc]"`
输出: `"aaabcbc"`

易错点：

数字有可能是两位数以上

括号间存在嵌套 因此需要用到栈

```
def decodeString(self, s: str) -> str:
    countstack = []
    cur = ""
    strstack = []
    k=0
    for ch in s:
        if ord('0')<=ord(ch)<=ord('9'):
            k = k*10 + ord(ch)-ord('0')
        elif ch == '[':
            countstack.append(k)
            k=0
            strstack.append(cur)
            cur=""
        elif ch == ']':
            k = countstack.pop()
            cur = k*cur
            k=0
            cur = strstack.pop()+cur
        else:
            cur+=ch
    return cur
```

72. 每日温度

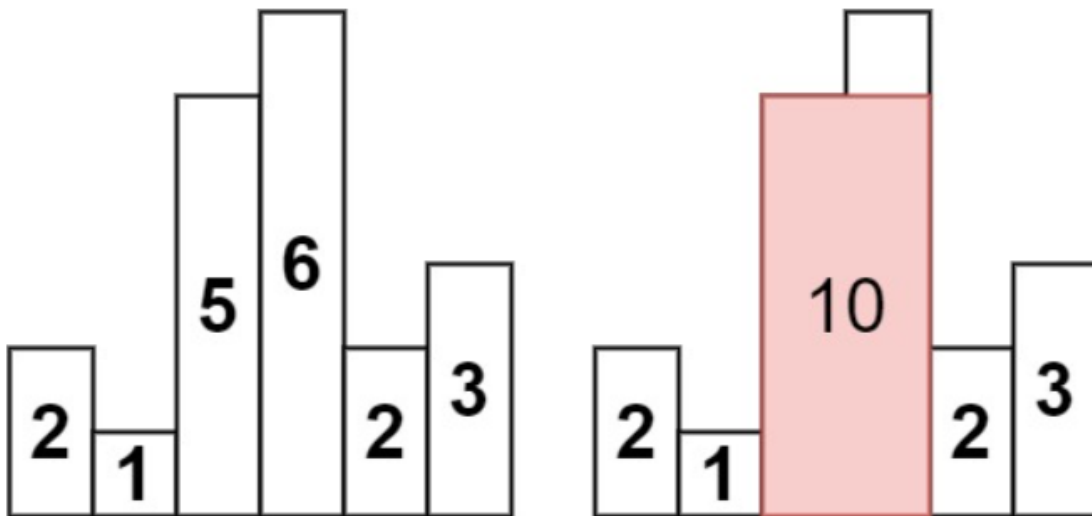
给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

输入: `temperatures = [73,74,75,71,69,72,76,73]`
输出: `[1,1,4,2,1,1,0,0]`

简单的单调栈

```
def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
    sstack = []
    n = len(temperatures)
    ans = [0]*n
    for i in range(n):
        while len(sstack)>0 and temperatures[i]>temperatures[sstack[-1]]:
            t = sstack.pop()
            ans[t] = i-t
        sstack.append(i)
    return ans
```

73. 柱状图中最大的矩形###



输入: heights = [2,1,5,6,2,3]

输出: 10

解释: 最大的矩形为图中红色区域, 面积为 10

```
def largestRectangleArea(self, heights: List[int]) -> int:
    sstack = []
    ans = 0
    heights.append(0)
    sstack.append(0)
    n = len(heights)
    for i in range(1, n):
        while len(sstack)!=0 and heights[i]<=heights[sstack[-1]]:
            t = sstack.pop()
            # 确定l是找左边合适的位置 l的地方是比h[t]小的最右节点 l+1对应的就是比h[t]大的
            # 所以 i-l-1 即是高为h[t]时 可以有的底的长
            if len(sstack)==0:
                l = -1
            else:
                l = sstack[-1]
            ans = max(ans, (i-l-1)*heights[t])
            sstack.append(i)
    return ans
```

74. 数组中的第K个最大元素

补充:

快速排序

```
n = len(nums)
def partition(left, right):
    t = nums[left]
    while left < right:
        while left < right and nums[right] >= t:
            right-=1
        nums[left] = nums[right]
        while left < right and nums[left] <= t:
            left+=1
        nums[right] = nums[left]
```

```

        left+=1
        nums[right] = nums[left]
        nums[left] = t
        return left
def quicksort(left,right):
    if left < right:
        part = partition(left, right)
        quicksort(part+1,right)
        quicksort(left, part-1)

quicksort(0,n-1)

```

```

# 简易版代码
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]
    return quicksort(left) + [pivot] + quicksort(right)

sorted_arr = quicksort(arr)

```

堆排序

```

def maxheapify(i , heapsize):
    l = i*2+1
    r = i*2+2
    ma = i
    if l<heapsize and nums[l] > nums[ma]:
        ma = l
    if r<heapsize and nums[r] > nums[ma]:
        ma = r
    if ma != i:
        nums[i], nums[ma] = nums[ma], nums[i]
        maxheapify(ma, heapsize)
def buildMaxHeap(heapsize):
    for i in range(heapsize//2 -1, -1,-1):
        maxheapify(i,heapsize)

buildMaxHeap(n) # 之后每次将头部与尾部交换 然后从上到下maxheapify

```

75. 前K个高频元素

输入: nums = [1,1,1,2,2,3], k = 2
输出: [1,2]

```

def topKFrequent(self, nums: List[int], k: int) -> List[int]:
    count = collections.Counter(nums) # 省的遍历统计得到字典
    count_arr = list(count.items())# 转化为列表
    heapsize = len(count_arr)
    ans = []
    def maxheapify(i):
        ma = i

```

```

l = 2*i+1
r = 2*i+2
if l < heapsize and count_arr[i][1] < count_arr[l][1]:
    ma = l
if r < heapsize and count_arr[ma][1] < count_arr[r][1]:
    ma = r
if ma != i:
    count_arr[i], count_arr[ma] = count_arr[ma], count_arr[i]
    maxheapify(ma)
def buildheapify():
    for i in range(heapsize//2 - 1, -1, -1):
        maxheapify(i)
buildheapify()
for i in range(k):
    ans.append(count_arr[0][0])
    count_arr[0], count_arr[heapsize-1] = count_arr[heapsize-1], count_arr[0]
    heapsize -= 1
    maxheapify(0)
return ans

```

76. 数据流的中位数

数据流 5 2 4 3 1 6

findMedian()

$$(4 + 3) / 2 = 3.5$$



堆顶

小顶堆 A

存储较大的一半



堆顶

大顶堆 B

存储较小的一半

```

def __init__(self):
    self.A = []
    self.B = []

def addNum(self, num: int) -> None:
    # A 小顶堆 存储数组中较大的一半
    # B 大顶堆 存储数组中较小的一半
    # 时刻要维持A比B多0/1个
    if len(self.A) == len(self.B):
        heappush(self.B, -num)
        heappush(self.A, -heappop(self.B))
    else:

```

```

        heappush(self.A, num)
        heappush(self.B, -heappop(self.A))

    def findMedian(self) -> float:
        if len(self.A) == len(self.B):
            return (self.A[0]-self.B[0])/2
        else:
            return self.A[0]

```

77. 买卖股票的最佳时机

输入: [7,1,5,3,6,4]

输出: 5

解释: 在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 注意利润不能是 7-1 = 6，因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

贪心算法

```

def maxProfit(self, prices: List[int]) -> int:
    ans = 0
    n = len(prices)
    curr = prices[0]
    for i in range(1, n):
        if prices[i]-curr < 0:
            curr = prices[i]
        ans = max(ans, prices[i]-curr)
    return ans

```

78. 跳跃游戏

输入: nums = [2,3,1,1,4]

输出: true

解释: 可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

```

def canJump(self, nums: List[int]) -> bool:
    curr = nums[0]
    n = len(nums)
    for i in range(n):
        if i <= curr: # 先判断能不能过去
            curr = max(curr, i+nums[i])
            if curr >= n-1:
                return True
    return False

```

79. 跳跃游戏 II####

输入: nums = [2,3,1,1,4]

输出: 2 返回跳到最后的最少次数

解释: 跳到最后一个位置的最小跳跃数是 2

从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置

```
def jump(self, nums: List[int]) -> int:
    maxPos = 0
    end = 0
    ans = 0
    n = len(nums)
    for i in range(n-1):
        maxPos = max(maxPos, i + nums[i])
        if i==end:
            end = maxPos
            ans+=1
    return ans
```

80. 划分字母区间

输入: s = "ababcbacadefegdehijhklij"

输出: [9,7,8]

解释: 划分结果为 "ababcbaca"、"defegde"、"hijhklij" 。

每个字母最多出现在一个片段中。

像 "ababcbacadefegde", "hijhklij" 这样的划分是错误的, 因为划分的片段数较少。

和上一题相似都是在贪心的选择结束位置end

```
def partitionLabels(self, s: str) -> List[int]:
    dic = {}
    n = len(s)
    for i in range(n):
        dic[s[i]] = i
    ans = []
    end = 0
    start = 0
    for i in range(n):
        end = max(end, dic[s[i]])
        if i == end:
            ans.append(end-start+1)
            start = i+1
    return ans
```

81. 爬楼梯

输入: n = 3

输出: 3

解释: 有三种方法可以爬到楼顶。

1. 1 阶 + 1 阶 + 1 阶
2. 1 阶 + 2 阶
3. 2 阶 + 1 阶

都走几步就能看出规律 n处的个数 = n-1 处的 加上 n-2处的

82. 杨辉三角

83. 打家劫舍

你是一个专业的小偷, 计划偷窃沿街的房屋。每间房内都藏有一定的现金, 影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统, **如果两间相邻的房屋在同一晚上被小偷闯入, 系统会自动报警。**

给定一个代表每个房屋存放金额的非负整数数组，计算你 **不触动警报装置的情况下**，一夜之内能够偷窃到的最高金额。

输入: [1,2,3,1]

输出: 4

解释: 偷窃 1 号房屋 (金额 = 1) ，然后偷窃 3 号房屋 (金额 = 3)。
偷窃到的最高金额 = 1 + 3 = 4 。

当前位置的最优值 = 上一个位置的值，上上一个位置的值+nums[i] 两者的最大值 分别代表了偷这家和不偷这家

```
def rob(self, nums: List[int]) -> int:
    n = len(nums)
    dp = [0]*(n+1)
    dp[1] = nums[0]
    for i in range(1,n):
        dp[i+1] = max(dp[i], dp[i-1]+nums[i])
    return dp[-1]
```

84. 完全平方数

输入: n = 12

输出: 3

解释: 12 = 4 + 4 + 4

注意一个易错点：决定这个数的分解 不一定是离他最近的平方数

例如: 12 = 9+dp[3] or 12 = 4+dp[8] 后者才是最优解

```
def numSquares(self, n: int) -> int:
    dp = [0]*(n+1)
    for i in range(1,n+1):
        temp = math.floor(math.sqrt(i))
        t = float("inf")
        for j in range(1,temp+1):
            t = min(t, dp[i-j*j])
        dp[i] = 1+t
    return dp[-1]
```

85. 零钱兑换

输入: coins = [1, 2, 5], amount = 11

输出: 3

解释: 11 = 5 + 5 + 1

```
def coinChange(self, coins: List[int], amount: int) -> int:
    dp = [float("inf")]*(amount+1)
    dp[0] = 0
    for i in range(1,amount+1):
        t = float("inf")
        for j in coins:
            if i-j>=0:
                t = min(t, dp[i-j]+1)
        dp[i] = t
    if dp[-1]==float("inf"):
        return -1
    else:
        return dp[-1]
```

86. 单词拆分

输入: s = "catsanddog", wordDict = ["cats", "dog", "sand", "and", "cat"]
输出: false

```
def wordBreak(self, s: str, wordDict: List[str]) -> bool:
    n = len(s)
    dp = [False]*(n+1)
    dp[0] = True
    for i in range(1,n+1):
        for j in range(i):
            if dp[j]==True and s[j:i] in wordDict:
                dp[i] = True
    return dp[-1]
```

87. 最长递增子序列

输入: nums = [10,9,2,5,3,7,101,18]
输出: 4
解释: 最长递增子序列是 [2,3,7,101]，因此长度为 4。

法一: 时间复杂度高 n^2

```
def lengthOfLIS(self, nums: List[int]) -> int:
    n = len(nums)
    dp = [1]*(n+1)
    ans = 1
    for i in range(1,n):
        for j in range(i-1,-1,-1):
            if nums[j] < nums[i]:
                dp[i+1] = max(dp[i+1],1 + dp[j+1])
        ans = max(ans,dp[i+1])
    return ans
```

88. 乘积最大子数组

输入: nums = [2,3,-2,4]
输出: 6
解释: 子数组 [2,3] 有最大乘积 6。

因为有负数的存在 没办法只靠前一个数的最大值来推断加上下一个数的最大值
所以我们增加当前位置的最大值和最小值来递推后面的值

```
def maxProduct(self, nums: List[int]) -> int:
    n = len(nums)
    ans = minnum = maxnum = nums[0]
    for i in range(1,n):
        if nums[i]>=0:
            minnum = min(nums[i]*minnum,nums[i])
            maxnum = max(nums[i]*maxnum,nums[i])
        else:
            t = minnum
            minnum = min(nums[i]*maxnum,nums[i])
            maxnum = max(nums[i]*t,nums[i])
        ans = max(ans,maxnum)
    return ans
```

89. 分割等和子集

补充: 0-1背包问题

dp[i][j]代表了 [0,i]的物品 放容量为j的背包下能获得的最大价值

列的循环须从后往前 从前往后就变成完全背包了

```
# 第一个是不放第i个物品 第二个是放第i个物品
dp[i][j] = max(dp[i-1][j], dp[i-1][j-weights[i]]+values[i])
# 一维 dp
for i in range(物品个数):
    for j in range(totalweight, weight[i]-1,-1):
        dp[j] = max(dp[j], dp[j-weights[i]]+values[i])
```

完全背包问题:

第二层循环从里向外 这样子就会保证物品可以重复放背包里

```
#完全背包中:
dp[i][j] = max(dp[i][j], dp[i][j - weights[i]] + values[i])
for i in range(物品个数):
    for j in range(weight[i], totalweight+1):
        dp[j] = max(dp[j], dp[j-weights[i]]+values[i])
```

怎么倒推path?

有二维dp 可以从右下角开始 逐行判断 如果最后一行不能放下该物品 则跳上一行

if dp[i][j] == dp[i][j-w[i]]+v[i]: 说明加入一次i 下一次判断重量j-w[i]

分割等和子集代码:

先确定和的一半 也就是背包问题的容量

这里每个值 既是价值又是容量

```
def canPartition(self, nums: List[int]) -> bool:
    he = sum(nums)
    n = len(nums)
    target = 0
    if he%2==1:
```

```

        return False
    else:
        target = he//2
        dp = [0]*(target+1)
        for i in range(n):
            for j in range(target, nums[i]-1,-1):
                dp[j] = max(dp[j], dp[j-nums[i]]+nums[i])
        if dp[target]==target:
            return True
        else:
            return False

```

90. 最长有效括号

输入: s = "()()())"

输出: 4

解释: 最长有效括号子串是 "()()"

动态规划

无外乎几种情况

(: 跳过 dp默认为0

) :() dp[i]= dp[i-2]+2

...((...)) dp[i] = dp[i-1]+dp[i-dp[i-1]-2]+2

```

def longestValidParentheses(self, s: str) -> int:
    n = len(s)
    dp = [0]*(n+1)
    maxans = 0
    for i in range(1,n):
        if s[i]==')':
            if s[i-1]=='(':
                if i-2>=0:
                    dp[i]= dp[i-2]+2
                else:
                    dp[i]=2
            else:
                if i-dp[i-1]-1>=0 and s[i-dp[i-1]-1]=='(':
                    dp[i] = dp[i-1]+dp[i-dp[i-1]-2]+2
                maxans = max(maxans,dp[i])
    return maxans

```

91. 不同路径



输入: $m = 3, n = 7$

输出: 28

动规 或者排列组合的算法

```
def uniquePaths(self, m: int, n: int) -> int:
    dp = [[1]*n for _ in range(m)]
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = dp[i-1][j]+dp[i][j-1]
    return dp[m-1][n-1]
```

92. 最小路径和

1	3	1
1	5	1
4	2	1

输入: `grid = [[1,3,1],[1,5,1],[4,2,1]]`

输出: 7

解释: 因为路径 $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$ 的总和最小。

```
def minPathSum(self, grid: List[List[int]]) -> int:
    m = len(grid)
    n = len(grid[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for i in range(1, n):
        dp[0][i] = grid[0][i]+dp[0][i-1]
    for i in range(1, m):
        dp[i][0] = grid[i][0] + dp[i-1][0]
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = min(dp[i-1][j],dp[i][j-1])+grid[i][j]
    return dp[m-1][n-1]
```

93. 最长回文子串

输入: s = "babad"

输出: "bab"

解释: "aba" 同样是符合题意的答案。

构建回文dp

```
def longestPalindrome(self, s: str) -> str:
    m = n = len(s)
    dp = [[True]*m for _ in range(n)]
    for j in range(1, m):
        for i in range(j):
            dp[i][j] = s[i]==s[j] and dp[i+1][j-1]
    ans = 0
    ss = ""
    for i in range(m):
        for j in range(n):
            if dp[i][j] == True and j-i+1>ans:
                ans = j-i+1
                ss = s[i:j+1]
    return ss
```

94. 最长公共子序列\$\$\$\$

例如, "ace" 是 "abcde" 的子序列, 但 "aec" 不是 "abcde" 的子序列。

输入: text1 = "abcde", text2 = "ace"

输出: 3

解释: 最长公共子序列是 "ace" , 它的长度为 3 。

```
def longestCommonSubsequence(self, text1: str, text2: str) -> int:
    m = len(text1)
    n = len(text2)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(1,n+1):
        for j in range(1, m+1):
            if text1[j-1] == text2[i-1]:
                dp[i][j] = dp[i-1][j-1]+1
            else:
                dp[i][j] = max(dp[i][j-1],dp[i-1][j])
    return dp[n][m]
```

95. 编辑距离

输入: word1 = "horse", word2 = "ros"
 输出: 3
 解释:
 horse -> rorse (将 'h' 替换为 'r')
 rorse -> rose (删除 'r')
 rose -> ros (删除 'e')

```
def minDistance(self, word1: str, word2: str) -> int:
    l1 = len(word1)
    l2 = len(word2)
    dp = [[0]*(l1+1) for _ in range(l2+1)]
    for i in range(1,l1+1):
        dp[0][i] = 1+dp[0][i-1]
    for j in range(1,l2+1):
        dp[j][0] = 1+dp[j-1][0]
    for i in range(1,l2+1):
        for j in range(1,l1+1):
            if word2[i-1] == word1[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j-1],dp[i-1][j],dp[i][j-1])
    return dp[l2][l1]
```

96. 只出现一次的数字

输入: nums = [2,2,1]
 输出: 1

```
def singleNumber(self, nums: List[int]) -> int:
    # 使用异或的性质 同假异真
    ans = 0
    for i in nums:
        ans = ans^i
    return ans
```

97. 多数元素

给定一个大小为 n 的数组 `nums`，返回其中的多数元素。多数元素是指在数组中出现次数 **大于** $\lfloor n/2 \rfloor$ 的元素。

输入: nums = [2,2,1,1,1,2,2]
输出: 2

```
def majorityElement(self, nums: List[int]) -> int:
    # 此算法被用来查找字符串中出现次数超一半的字符
    candidate = 0
    count = 0
    for i in nums:
        if count==0:
            candidate = i
            count = 1
        else:
            if candidate==i:
                count+=1
            else:
                count-=1
    return candidate
```

98. 颜色分类

输入: nums = [2,0,2,1,1,0]
输出: [0,0,1,1,2,2]

P0保证左面都是0 p2保证右面都是2

```
def sortColors(self, nums: List[int]) -> None:
    n = len(nums)
    p0 = 0
    p2 = n-1
    for i in range(n):
        while i<=p2 and nums[i] == 2:
            nums[i],nums[p2] = nums[p2],nums[i]
            p2-=1
        if nums[i] == 0:
            nums[p0],nums[i] = nums[i],nums[p0]
            p0+=1
        if i>p2:
            break
```

99. 下一个排列

输入: nums = [1,2,3]
输出: [1,3,2]

P0代表要交换的位置 第一个比左边位置大的

1 2 5 4 3

p0该指向2的位置 找p1 指向的是第一个比p0位置大的值

两者交换后 p0后面的列表依旧可以保持 降序 这时候只要将其反转即可找到结果

```
def nextPermutation(self, nums: List[int]) -> None:
    n = len(nums)
    p0 = -1 #初值设为-1 为了处理完全倒序 5, 4, 3, 2, 1的情况
```

```

p1 = 0
for i in range(n-1,0,-1):
    if nums[i] > nums[i-1]:
        p0 = i-1
        break
for i in range(n-1,0,-1):
    if p0>=0 and nums[i] > nums[p0]:
        p1 = i
        nums[p0],nums[p1] = nums[p1],nums[p0]
        break
left = p0+1
right = n-1
while left<right:
    nums[left],nums[right] = nums[right],nums[left]
    left+=1
    right-=1

```

100. 寻找重复数 重点

输入: nums = [1,3,4,2,2]
输出: 2

指针既是指向 又是当前位置的值

```

def findDuplicate(self, nums: List[int]) -> int:
    n = len(nums)
    # 快慢指针
    low = nums[0]
    fast = nums[nums[0]]
    while low!=fast:
        low = nums[low]
        fast = nums[nums[fast]]
    fast = 0
    while low!=fast:
        low = nums[low]
        fast = nums[fast]
    return fast

```